

Nama:Hafidza Tsamara Zahra
NIM:254107020034
Kelas:1G-TI
Absen:11

LAPORAN TUGAS JOBSHEET 12

12.2 Percobaan 1: Operasi Penambahan pada Double Linked List

class mahasiswa11.java

```
package jobsheet12;

public class mahasiswa11 {
    String nim;
    String nama;
    String kelas;
    double ipk;

    public mahasiswa11(String nim, String nama, String kelas, double ipk) {
        this.nim = nim;
        this.nama = nama;
        this.kelas = kelas;
        this.ipk = ipk;
    }

    public void tampil() {
        System.out.println(
            "NIM      : " + nim +
            "\nNama    : " + nama +
            "\nKelas   : " + kelas +
            "\nIPK      : " + ipk
        );
    }
}
```

class node11.java

```
package jobsheet12;

public class node11 {
    mahasiswa11 data;
    node11 prev;
    node11 next;
}
```

```

    public node11(mahasiswa11 data) {
        this.data = data;
        this.prev = null;
        this.next = null;
    }
}

```

class doubleLinkedList11.java

```

package jobsheet12;

public class doubleLinkedList11 {
    node11 head;
    node11 tail;

    public doubleLinkedList11() {
        head = null;
        tail = null;
    }

    public boolean isEmpty() {
        return head == null;
    }

    public void addFirst(mahasiswa11 data) {
        node11 newNode = new node11(data);
        if (isEmpty()) {
            head = tail = newNode;
        } else {
            newNode.next = head;
            head.prev = newNode;
            head = newNode;
        }
    }

    public void addLast(mahasiswa11 data) {
        node11 newNode = new node11(data);
        if (isEmpty()) {
            head = tail = newNode;
        } else {
            tail.next = newNode;
            newNode.prev = tail;
            tail = newNode;
        }
    }
}

```

```

    }

    public void insertAfter(String keyNim, mahasiswa11 data) {
        node11 current = head;
        while (current != null && !current.data.nim.equals(keyNim)) {
            current = current.next;
        }

        if (current == null) {
            System.out.println("Data dengan NIM " + keyNim + " tidak
ditemukan.");
            return;
        }

        node11 newNode = new node11(data);

        if (current == tail) {
            newNode.prev = current;
            current.next = newNode;
            tail = newNode;
        } else {
            newNode.prev = current;
            newNode.next = current.next;
            current.next.prev = newNode;
            current.next = newNode;
        }

        System.out.println("Data berhasil disisipkan setelah NIM " +
keyNim);
    }

    public void print() {
        if (isEmpty()) {
            System.out.println("Linked List masih kosong.");
            return;
        }

        node11 current = head;
        while (current != null) {
            current.data.tampil();
            current = current.next;
        }
    }

    public void removeFirst() {

```

```

        if (isEmpty()) {
            System.out.println("Linked list kosong.");
        } else if (head == tail) {
            head = tail = null;
        } else {
            head = head.next;
            head.prev = null;
        }

        System.out.println("Data awal berhasil dihapus.");
    }

    public void removeLast() {
        if (isEmpty()) {
            System.out.println("Linked list kosong.");
        } else if (head == tail) {
            head = tail = null;
        } else {
            tail = tail.prev;
            tail.next = null;
        }

        System.out.println("Data akhir berhasil dihapus.");
    }
}

```

class doubleLinkedListMain11.java

```

package jobsheet12;

import java.util.Scanner;

public class doubleLinkedListMain11 {

    public static void main(String[] args) {

        Scanner scan = new Scanner(System.in);
        doubleLinkedList11 list = new doubleLinkedList11();

        int pilihan;

        do {

```

```

        System.out.println("\n===== MENU DOUBLE LINKED LIST
=====");

        System.out.println("1. Tambah data di awal");
        System.out.println("2. Tambah data di akhir");
        System.out.println("3. Sisipkan data di tengah (setelah
NIM) ");

        System.out.println("4. Hapus data di awal");
        System.out.println("5. Hapus data di akhir");
        System.out.println("6. Tampilkan data");
        System.out.println("0. Keluar");
        System.out.print("Pilih menu : ");

        pilihan = scan.nextInt();
        scan.nextLine();

        switch (pilihan) {

            case 1:
                mahasiswa11 mhsAwal = inputMahasiswa(scan);
                list.addFirst(mhsAwal);
                break;

            case 2:
                mahasiswa11 mhsAkhir = inputMahasiswa(scan);
                list.addLast(mhsAkhir);
                break;

            case 3:
                System.out.print("Masukkan NIM yang dicari : ");
                String keyNim = scan.nextLine();

                System.out.println("Masukkan data baru:");
                mahasiswa11 dataBaru = inputMahasiswa(scan);

                list.insertAfter(keyNim, dataBaru);
                break;

            case 4:
                list.removeFirst();
                break;

            case 5:
                list.removeLast();

```

```

        break;

        case 6:
            list.print();
            break;

        case 0:
            System.out.println("Program selesai.");
            break;

        default:
            System.out.println("Menu tidak valid.");
    }

    } while (pilihan != 0);

    scan.close();
}

public static mahasiswa11 inputMahasiswa(Scanner scan) {

    System.out.print("NIM    : ");
    String nim = scan.nextLine();

    System.out.print("Nama    : ");
    String nama = scan.nextLine();

    System.out.print("Kelas : ");
    String kelas = scan.nextLine();

    System.out.print("IPK    : ");
    double ipk = scan.nextDouble();
    scan.nextLine();

    return new mahasiswa11(nim, nama, kelas, ipk);
}
}

```

hasil output

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
2. Tambah data di akhir

3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu : 2
NIM : 123005
Nama : Harry
Kelas : 1A
IPK : 3.76

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu : 3
Masukkan NIM yang dicari : 123005
Masukkan data baru:
NIM : 123010
Nama : Potter
Kelas : 1B
IPK : 3.55
Data berhasil disisipkan setelah NIM 123005

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu : 6
NIM : 123005
Nama : Harry
Kelas : 1A
IPK : 3.76
NIM : 123010
Nama : Potter
Kelas : 1B
IPK : 3.55

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar

Pilih menu : 0
Program selesai.

12.2.3 Pertanyaan

1. Jelaskan perbedaan struktur dan mekanisme traversal antara Single Linked List dan Double Linked List!
2. Perhatikan class Node, di dalamnya terdapat atribut next dan prev. Jelaskan fungsi masing-masing atribut tersebut pada proses traversal dan manipulasi node!
3. Perhatikan konstruktor pada class DoubleLinkedList. Jelaskan fungsi konstruktor tersebut terhadap kondisi awal linked list!
4. Perhatikan potongan kode berikut:

```
if (isEmpty()) {  
    head = tail = newNode;  
}
```

Mengapa head dan tail harus menunjuk node yang sama ketika linked list masih kosong?

5. Modifikasi method print() agar menampilkan pesan "Linked List masih kosong" ketika tidak terdapat data pada linked list!
6. Modifikasi kode program dengan menambahkan method printReverse() untuk menampilkan seluruh data pada Double Linked List secara terbalik, dimulai dari node tail menuju head!

jawaban

1. Single Linked List memiliki satu pointer yaitu next yang hanya menunjuk ke node berikutnya. Traversal pada Single Linked List hanya dapat dilakukan dari depan ke belakang (satu arah).

Sedangkan Double Linked List memiliki dua pointer yaitu next dan prev. Pointer next menunjuk node berikutnya, sedangkan prev menunjuk node sebelumnya. Oleh karena itu traversal pada Double Linked List dapat dilakukan dua arah, yaitu dari depan ke belakang maupun dari belakang ke depan.

2. next berfungsi untuk menyimpan alamat node berikutnya sehingga traversal dapat bergerak maju ke node selanjutnya.

prev berfungsi untuk menyimpan alamat node sebelumnya sehingga traversal dapat bergerak mundur ke node sebelumnya.

3. Konstruktor pada class doubleLinkedList11 berfungsi untuk menginisialisasi linked list dalam keadaan kosong dengan memberikan nilai null pada head dan tail.

```
public doubleLinkedList11() {  
  
    head = null;  
  
    tail = null;  
}
```

```
}
```

Artinya saat objek linked list pertama kali dibuat, belum terdapat node/data di dalam linked list.

4. Karena ketika linked list masih kosong dan hanya terdapat satu node pertama, maka node tersebut sekaligus menjadi:
 - node pertama (head)
 - node terakhir (tail)

Oleh sebab itu head dan tail harus menunjuk ke node yang sama agar linked list memiliki awal dan akhir yang valid.

5.

```
public void print() {  
  
    if (isEmpty()) {  
  
        System.out.println("Linked List masih kosong.");  
  
        return;  
  
    }  
  
    node11 current = head;  
  
    while (current != null) {  
  
        current.data.tampil();  
  
        current = current.next;  
  
    }  
  
}
```

6. modifikasi class doubleLinkedList11.java

```
public void printReverse() {  
  
    if (isEmpty()) {  
  
        System.out.println("Linked List masih kosong.");  
  
        return;  
  
    }  
  
}
```

```

    }

    node11 current = tail;

    while (current != null) {

        current.data.tampil();

        current = current.prev;

    }

}

```

modifikasi class doubleLinkedListMain11.java

```
System.out.println("7. Tampilkan data reverse");
```

```

case 7:

    list.printReverse();

    break;

```

hasil output

```

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data reverse
0. Keluar
Pilih menu : 2
NIM  : 123005
Nama : Harry

```

Kelas : 1A
IPK : 3.76

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data reverse
0. Keluar

Pilih menu : 1

NIM : 123010

Nama : Potter

Kelas : 1B

IPK : 3.55

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data reverse
0. Keluar

Pilih menu : 3

Masukkan NIM yang dicari : 123005

Masukkan data baru:

NIM : 12345

Nama : Rara

Kelas : 1G

IPK : 3.75

Data berhasil disisipkan setelah NIM 123005

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data reverse
0. Keluar

Pilih menu : 6

NIM : 123010

Nama : Potter

Kelas : 1B

IPK : 3.55

NIM : 123005

Nama : Harry

Kelas : 1A

IPK : 3.76

```

NIM : 12345
Nama : Rara
Kelas : 1G
IPK : 3.75

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data reverse
0. Keluar
Pilih menu : 7
NIM : 12345
Nama : Rara
Kelas : 1G
IPK : 3.75
NIM : 123005
Nama : Harry
Kelas : 1A
IPK : 3.76
NIM : 123010
Nama : Potter
Kelas : 1B
IPK : 3.55

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data reverse
0. Keluar
Pilih menu : 0
Program selesai.

```

12.3 Percobaan 2: Operasi Penghapusan pada Double Linked List

class doubleLinkedList11.java

```

package jobsheet12;

public class doubleLinkedList11 {

    node11 head;

    node11 tail;

```

```
public doubleLinkedList11() {

    head = null;

    tail = null;

}

public boolean isEmpty() {

    return head == null;

}

public void addFirst(mahasiswa11 data) {

    node11 newNode = new node11(data);

    if (isEmpty()) {

        head = tail = newNode;

    } else {

        newNode.next = head;

        head.prev = newNode;

        head = newNode;

    }

}

public void addLast(mahasiswa11 data) {

    node11 newNode = new node11(data);

    if (isEmpty()) {

        head = tail = newNode;

    } else {
```

```

        tail.next = newNode;

        newNode.prev = tail;

        tail = newNode;

    }

}

public void insertAfter(String keyNim, mahasiswa11 data) {

    node11 current = head;

    while (current != null && !current.data.nim.equals(keyNim)) {

        current = current.next;

    }

    if (current == null) {

        System.out.println("Data dengan NIM " + keyNim + " tidak
ditemukan.");

        return;

    }

    node11 newNode = new node11(data);

    if (current == tail) {

        newNode.prev = current;

        current.next = newNode;

        tail = newNode;

    } else {

```

```
        newNode.prev = current;

        newNode.next = current.next;

        current.next.prev = newNode;

        current.next = newNode;

    }

    System.out.println("Data berhasil disisipkan setelah NIM " +
keyNim);

}

public void print() {

    if (isEmpty()) {

        System.out.println("Linked List masih kosong.");

        return;

    }

    node11 current = head;

    while (current != null) {

        current.data.tampil();

        current = current.next;

    }

}

public void removeFirst() {

    if (isEmpty()) {

        System.out.println("Linked List kosong.");
```

```
        return;

    }

    if (head == tail) {

        head = tail = null;

    } else {

        head = head.next;

        head.prev = null;

    }

    System.out.println("data berhasil dihapus");

}

public void removeLast() {

    if (isEmpty()) {

        System.out.println("Linked List kosong.");

        return;

    }

    if (head == tail) {

        head = tail = null;

    } else {

        tail = tail.prev;

        tail.next = null;

    }

    System.out.println("data berhasil dihapus");

}
```

```

    }

    public void printReverse() {

        if (isEmpty()) {

            System.out.println("Linked List masih kosong.");

            return;

        }

        node11 current = tail;

        while (current != null) {

            current.data.tampil();

            current = current.prev;

        }

    }
}

```

hasil output

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data reverse
0. Keluar

Pilih menu : 2

NIM : 123005
Nama : Harry
Kelas : 1A
IPK : 3.76

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data reverse
0. Keluar

Pilih menu : 3

Masukkan NIM yang dicari : 123005

Masukkan data baru:

NIM : 123010

Nama : Potter

Kelas : 1B

IPK : 3.55

Data berhasil disisipkan setelah NIM 123005

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data reverse
0. Keluar

Pilih menu : 5

data berhasil dihapus

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data reverse
0. Keluar

Pilih menu : 6

NIM : 123005

Nama : Harry

Kelas : 1A

IPK : 3.76

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)

4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data reverse
0. Keluar
Pilih menu : 0
Program selesai.

12.3.3 Pertanyaan Percobaan

1. Perhatikan potongan kode berikut pada method `removeFirst()`:

```
head = head.next;  
  
head.prev = null;
```

Jelaskan fungsi masing-masing statement tersebut pada proses penghapusan node!

2. Modifikasi method `removeFirst()` dan `removeLast()` agar program menampilkan data yang berhasil dihapus!

jawaban

- 1.

```
head = head.next;
```

Statement ini berfungsi untuk memindahkan posisi head ke node berikutnya sehingga node pertama sebelumnya tidak lagi menjadi bagian dari linked list.

```
head.prev = null;
```

Statement ini berfungsi untuk menghapus hubungan node baru dengan node sebelumnya agar node pertama yang baru tidak memiliki node sebelum dirinya. Dengan demikian struktur Double Linked List tetap benar.

2. modifikasi

```
public void removeFirst() {  
  
    if (isEmpty()) {  
  
        System.out.println("Linked List kosong.");  
  
        return;  
  
    }  
}
```

```
        mahasiswa11 dataHapus = head.data;

        if (head == tail) {

            head = tail = null;

        } else {

            head = head.next;

            head.prev = null;

        }

        System.out.println("Data yang dihapus:");

        dataHapus.tampil();

    }

    public void removeLast() {

        if (isEmpty()) {

            System.out.println("Linked List kosong.");

            return;

        }

        mahasiswa11 dataHapus = tail.data;

        if (head == tail) {

            head = tail = null;

        } else {

            tail = tail.prev;
```

```

        tail.next = null;

    }

    System.out.println("Data yang dihapus:");

    dataHapus.tampil();

}

```

hasil output

===== MENU DOUBLE LINKED LIST =====

```

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data reverse
0. Keluar
Pilih menu : 2
NIM  : 123005
Nama  : Harry
Kelas : 1A
IPK   : 3.76

```

===== MENU DOUBLE LINKED LIST =====

```

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data reverse
0. Keluar
Pilih menu : 3
Masukkan NIM yang dicari : 123005
Masukkan data baru:
NIM  : 123010
Nama  : Potter
Kelas : 1B
IPK   : 3.55
Data berhasil disisipkan setelah NIM 123005

```

===== MENU DOUBLE LINKED LIST =====

```
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data reverse
0. Keluar
Pilih menu : 5
Data yang dihapus:
NIM   : 123010
Nama  : Potter
Kelas : 1B
IPK   : 3.55

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data reverse
0. Keluar
Pilih menu : 6
NIM   : 123005
Nama  : Harry
Kelas : 1A
IPK   : 3.76

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data reverse
0. Keluar
Pilih menu : 0
Program selesai.
```

PROGRES CM

```
class pembeli11.java
package jobsheet12;
```

```

public class pembeli11 {
    int noAntrian;
    String namaPembeli;
    String noHp;

    public pembeli11(int noAntrian, String namaPembeli, String noHp) {
        this.noAntrian = noAntrian;
        this.namaPembeli = namaPembeli;
        this.noHp = noHp;
    }

    public void tampil() {
        System.out.println(noAntrian + "\t" + namaPembeli + "\t" +
noHp);
    }
}

```

class nodePembeli11.java

```

package jobsheet12;

public class nodePembeli11 {
    pembeli11 data;
    nodePembeli11 prev;
    nodePembeli11 next;

    public nodePembeli11(pembeli11 data) {
        this.data = data;
        prev = null;
        next = null;
    }
}

```